Ain Shams University
Faculty of Engineering

PROJECT REPORT

3-DOF Parallel ROBOT (Delta-Type)

MCT442: Advanced Robotics Analysis and Control

Ahmed Ahmed Reda 23p0151
Karim Magdy atef 23p0068
Ahmed Khaled 23p0364
Mohamed Ashraf 23p0377
Ahmed Hatem 23p0322

Contribution table:

Ahmed Khaled	**Mechanical and PID control**
Mohamed Ashraf	**Matlab**
Ahmed Ahmed Reda	**Electrical**
Ahmed Hatem	**Matlab and CAD**
Karim magdy	**Soft gripper**

Executive Summary

This project presents the design, manufacture, and control of a 3-DOF parallel translational robot of the Delta type.

The project is structured across three milestones. The first milestone focuses on MATLAB simulation, where the robot's CAD model is imported and both Forward Kinematics (FK) and Inverse Kinematics (IK) models are implemented and validated through point-to-point and path-following motion simulations. The second milestone covers physical manufacturing and hardware control: Joint 1 is driven by a DC motor with an encoder under a cascaded PID control scheme, while Joints 2 and 3 use servo or stepper motors for simplicity. Two software control modes are implemented — a Forward Kinematics mode where the user inputs joint angles and the end-effector position is computed and validd against physical measurements, and an Inverse Kinematics mode where the user inputs a desired Cartesian position and the robot moves accordingly, with achieved position logged and compared. Performance is evaluated through FK accuracy, IK positioning accuracy, FK/IK consistency, repeatability, and joint tracking metrics (settling time, overshoot, and steady-state error). The third milestone integrates a lightweight soft gripper with binary open/close control to demonstrate a full pick-and-place operation.

Mechanical Part

1) Cad Design

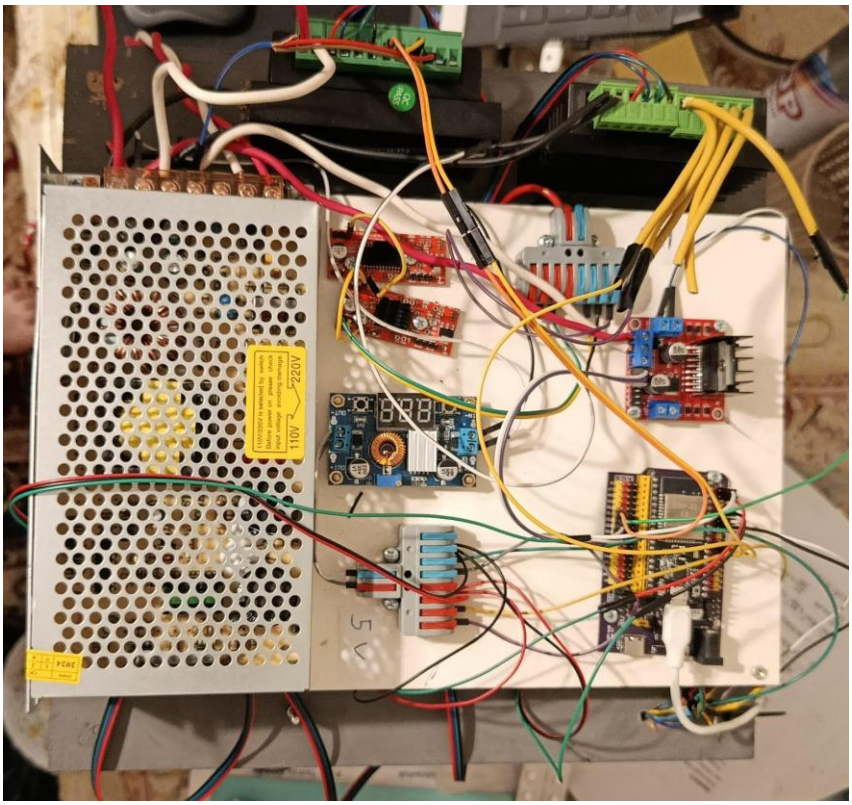
The Delta robot was modeled in CAD as a full 3D assembly comprising all structural and mechanical components. The design consists of three identical kinematic chains, each connecting the fixed upper base platform to the mobile end-effector platform through a pair of parallel linkage rods, forming the characteristic Delta parallel structure.





2) Electrical

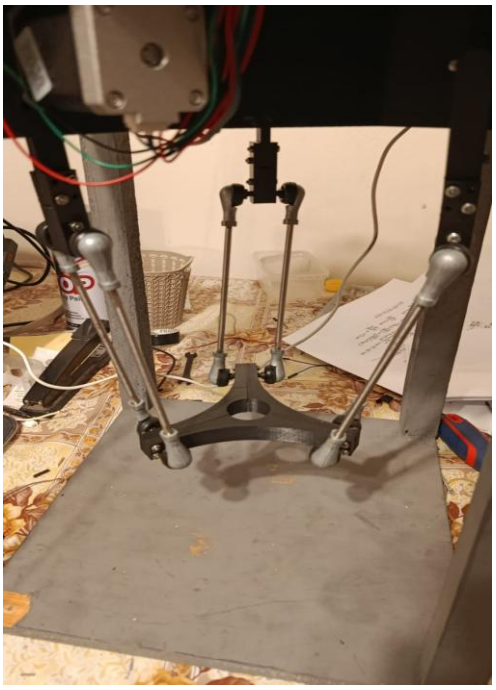
By using two stepper motors to guide to links and a DC-motor to guide the last link and to make a the cascaded PID, Using a two stepper motors deriver TB6600 to drive motors by 1/16 micro steps and H bridge guiding the dc motor for directions



3) The Robot Assembly

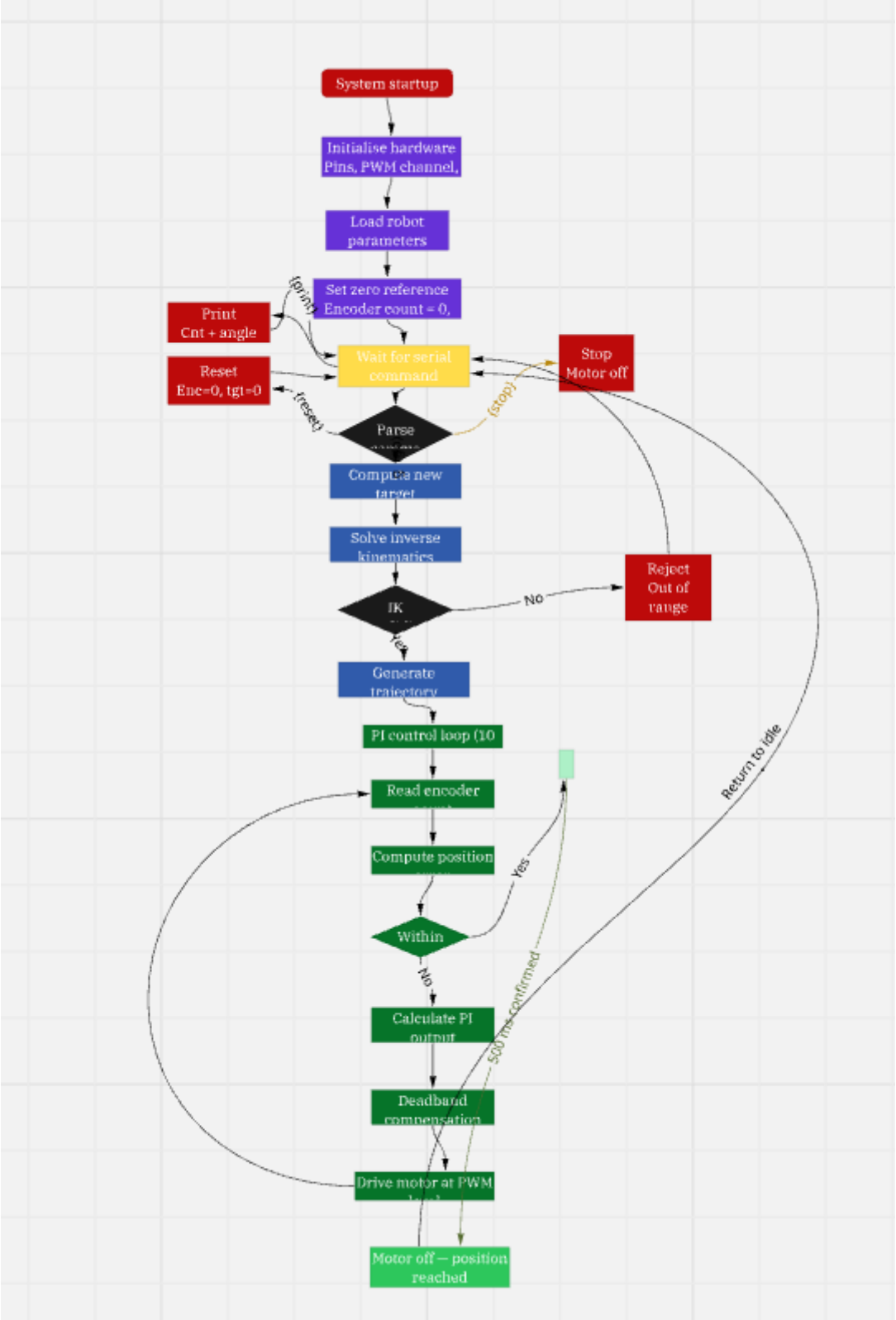
All cad parts where printed using 3D printing to be used in this and assembled rods and links was made in aluminium to give us this final product





Diagrams

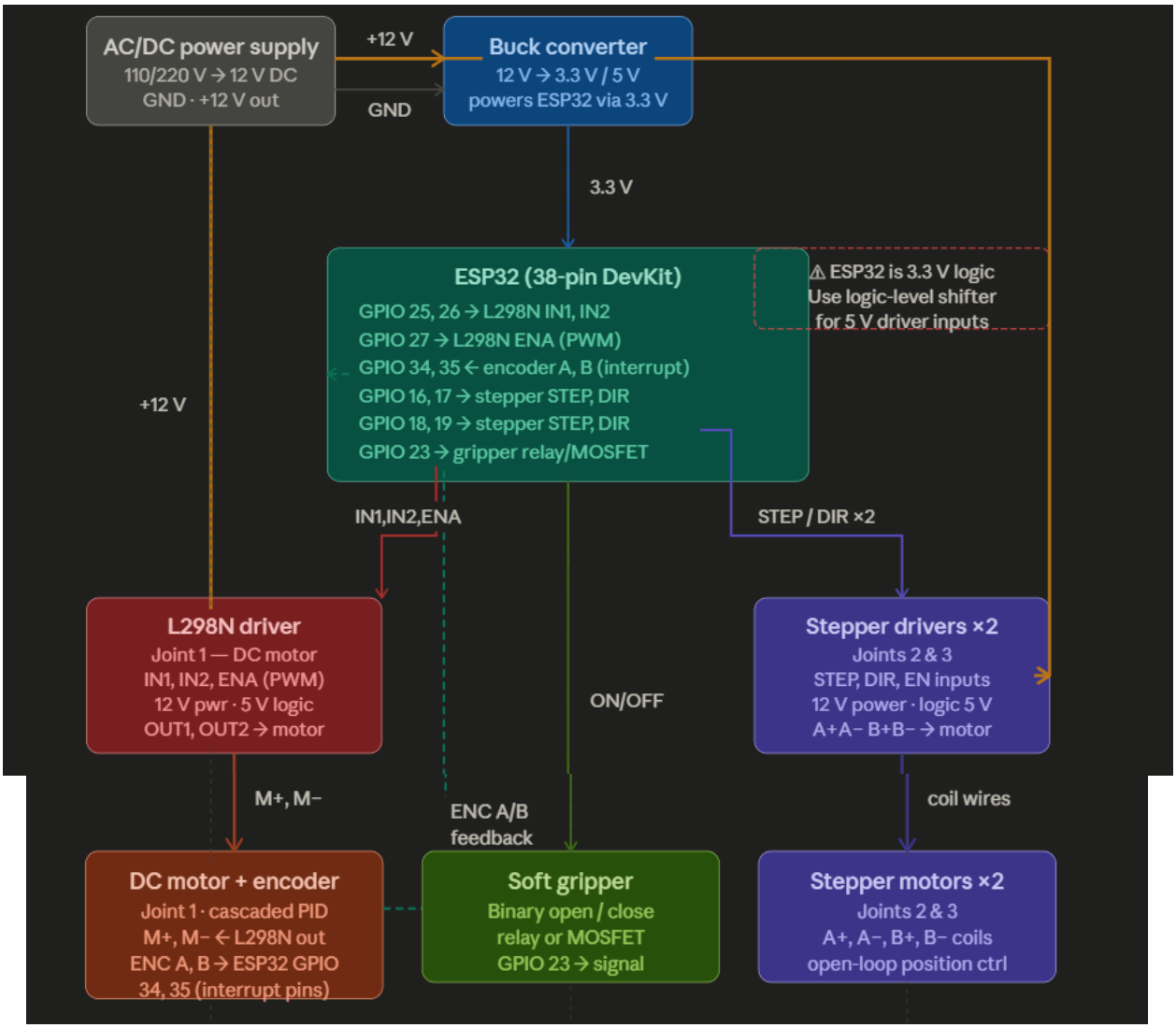
1) FlowChart:



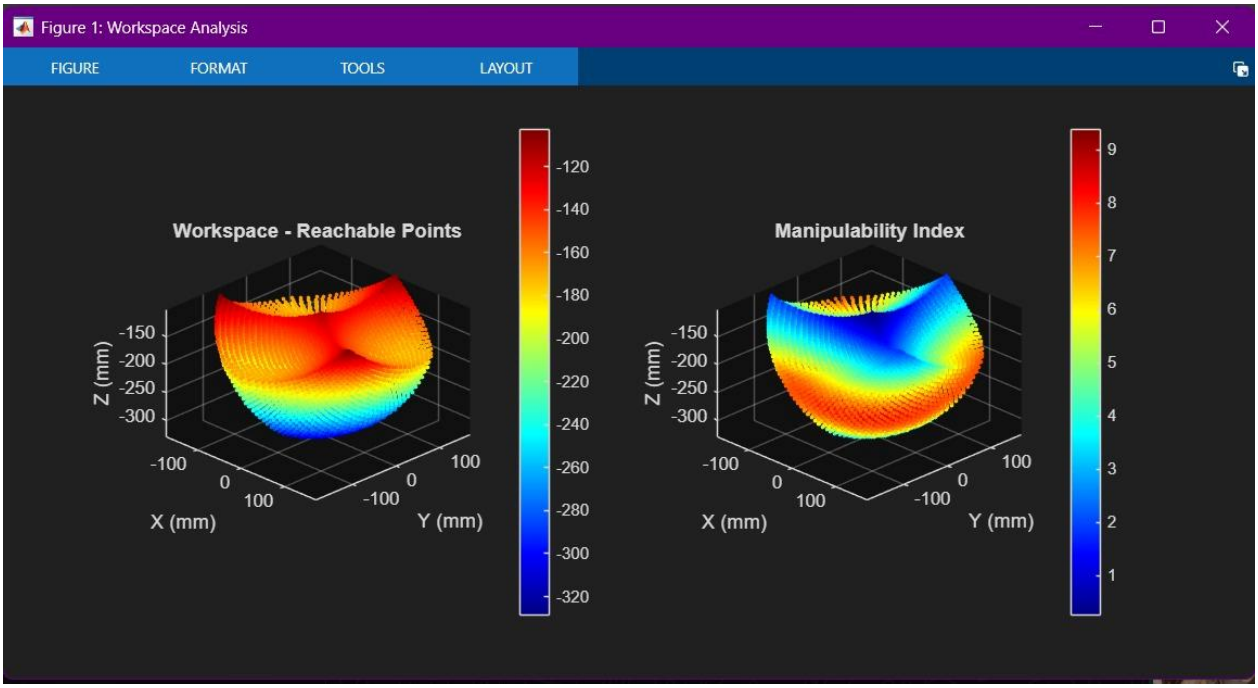
2) State Diagram



3) wiring diagram



Work space and manipubility



Workspace Reachable Points: The reachable workspace forms a dome-shaped volume spanning approximately ±100 mm in both the X and Y axes, with a Z range from roughly −120 mm to −320 mm (measured downward from the base frame). The color gradient represents the Z-height of each reachable point, transitioning from red (highest, near −120 mm) to blue (lowest, near −320 mm). The characteristic concave depression at the center top of the workspace is typical of Delta-type robots, caused by mechanical interference limits near the singularity region directly beneath the base platform.

Manipulability Index: The manipulability index quantifies how freely the end-effector can move at each point within the workspace. Higher index values (red, up to ~9) appear near the outer upper rim of the workspace, indicating directions where the robot has greater dexterity and force transmission capability. Lower values (blue, ~1) are concentrated toward the bottom and the central depression, corresponding to near-singular configurations where the robot loses maneuverability. This analysis is critical for path planning, as trajectories should ideally be confined to regions of high manipulability to ensure smooth and accurate motion.

The code of work space and manipulability:

```
function WorkspaceAnalysis()
%%
=========================================================
%   WORKSPACE ANALYSIS - Delta Robot
%   - Plot all reachable points (point cloud)
%   - Calculate Manipulability Index at each point
%%
=========================================================

clear; clc; close all;
global r R l L

% Robot Parameters
R = 107.5;
r = 32.5;
L = 100;
l = 180;

fprintf('=== Workspace Analysis ===\n');
fprintf('Scanning joint space...\n');

%% STEP 1: Scan all joint angle combinations
% theta range: typical delta robot -30 to 90 degrees
th_range = -30:5:90;   % step = 5 deg (simple & fast)

% Store reachable points
pts   = [];   % [X Y Z]
manip = [];   % manipulability index

count = 0;

for th1 = th_range
   for th2 = th_range
      for th3 = th_range

         % Forward Kinematics
         [X, Y, Z, fl] = FKinem(th1, th2, th3);

         if fl == 0   % point is reachable
            count = count + 1;
```

```
            pts(count, :) = [X, Y, Z];

            % Manipulability Index
            mu = calcManipulability(th1, th2, th3);
            manip(count) = mu;
          end

      end
   end
end

fprintf('Total reachable points found: %d\n\n', count);

%% STEP 2: Plot Workspace (point cloud colored by height Z)
figure('Name', 'Workspace Analysis', 'Position', [100 100 1200 500]);

subplot(1,2,1);
scatter3(pts(:,1), pts(:,2), pts(:,3), ...
        5, pts(:,3), 'filled');    % color = Z value
colorbar;
colormap(jet);
xlabel('X (mm)'); ylabel('Y (mm)'); zlabel('Z (mm)');
title('Workspace - Reachable Points');
grid on; axis equal;
view(45, 25);

%% STEP 3: Plot Manipulability Index (color = manipulability)
subplot(1,2,2);
scatter3(pts(:,1), pts(:,2), pts(:,3), ...
        5, manip, 'filled');    % color = manipulability
colorbar;
colormap(jet);
xlabel('X (mm)'); ylabel('Y (mm)'); zlabel('Z (mm)');
title('Manipulability Index');
grid on; axis equal;
view(45, 25);

%% STEP 4: Print Statistics
fprintf('=== Manipulability Statistics ===\n');
fprintf('Max  Manipulability = %.4f\n', max(manip));
fprintf('Min  Manipulability = %.4f\n', min(manip));
fprintf('Mean Manipulability = %.4f\n', mean(manip));
```

```
fprintf('\nWorkspace Limits:\n');
fprintf('X: [%.1f , %.1f] mm\n', min(pts(:,1)), max(pts(:,1)));
fprintf('Y: [%.1f , %.1f] mm\n', min(pts(:,2)), max(pts(:,2)));
fprintf('Z: [%.1f , %.1f] mm\n', min(pts(:,3)), max(pts(:,3)));

end


%%
% ============================================================
%   Helper: Calculate Manipulability Index
%   mu = sqrt(det(J * J'))
%   Using numerical Jacobian (simple finite difference)
%%
% ============================================================
function mu = calcManipulability(th1, th2, th3)

    delta = 0.01;   % small perturbation in degrees

    % Current position
    [X0, Y0, Z0, fl0] = FKinem(th1, th2, th3);
    if fl0 ~= 0; mu = 0; return; end

    % Partial derivatives (numerical)
    [X1, Y1, Z1, fl1] = FKinem(th1+delta, th2, th3);
    [X2, Y2, Z2, fl2] = FKinem(th1, th2+delta, th3);
    [X3, Y3, Z3, fl3] = FKinem(th1, th2, th3+delta);

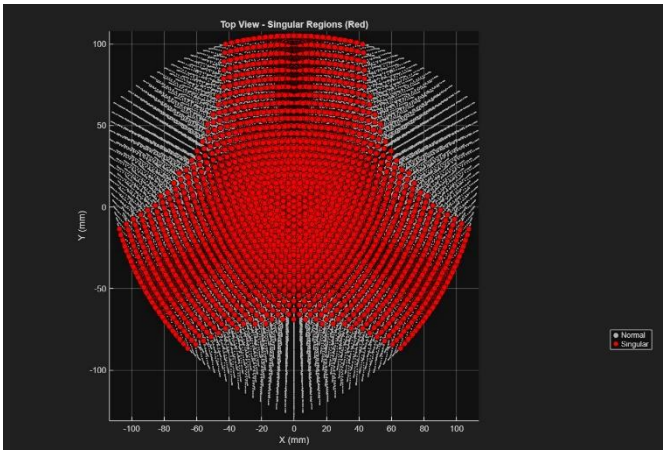
    if fl1~=0 || fl2~=0 || fl3~=0
        mu = 0; return;
    end

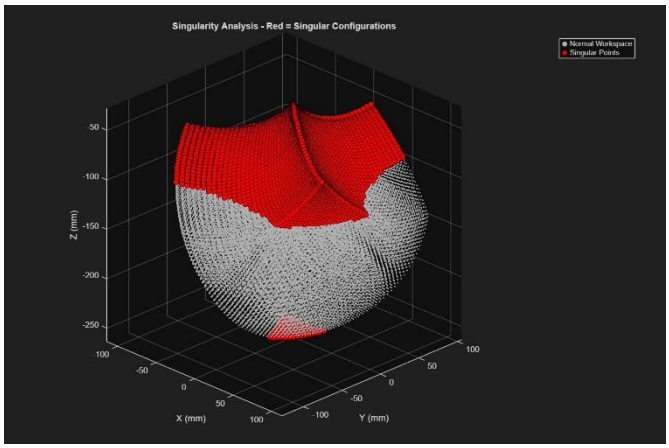
    % Jacobian columns = dX/dth, dY/dth, dZ/dth
    J = [ (X1-X0)/delta, (X2-X0)/delta, (X3-X0)/delta ;
          (Y1-Y0)/delta, (Y2-Y0)/delta, (Y3-Y0)/delta ;
          (Z1-Z0)/delta, (Z2-Z0)/delta, (Z3-Z0)/delta ];

    % Manipulability = sqrt(det(J*J'))
    mu = sqrt(abs(det(J * J')));

end
```

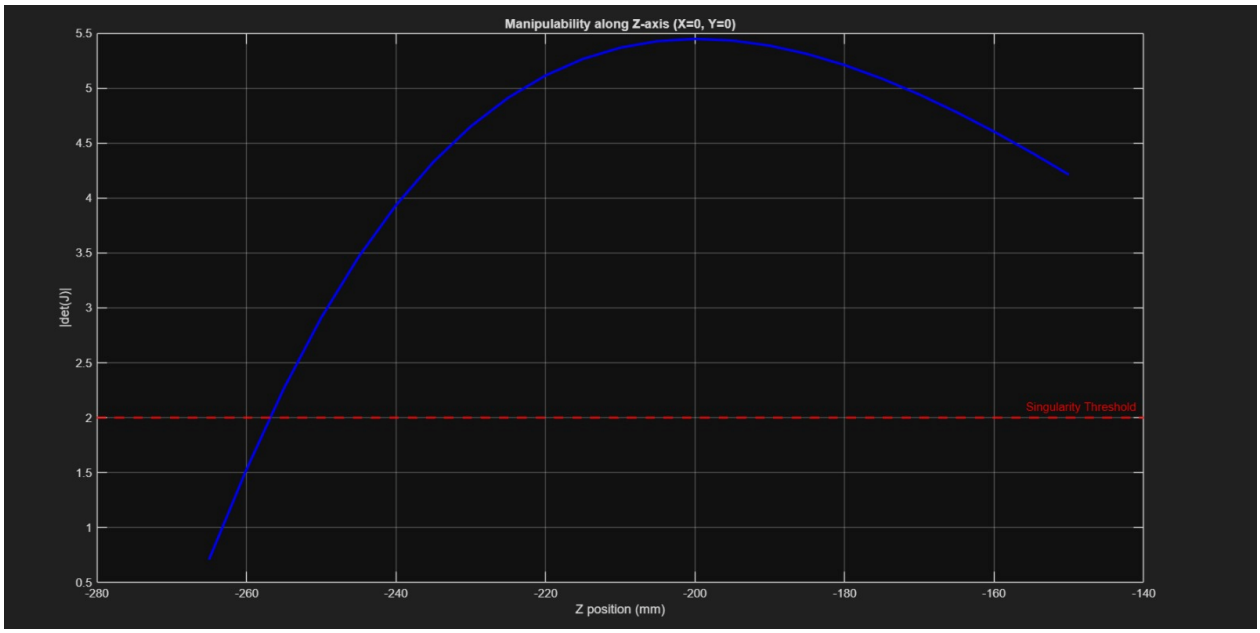
Singularity analysis





To ensure safe and reliable operation of the Delta robot, a comprehensive singularity analysis was performed in MATLAB by evaluating the Jacobian determinant |det(J)| across the entire reachable workspace. Singular configurations — where the robot loses controllability or experiences infinite joint velocities for finite end-effector motion — were identified using a defined singularity threshold of |det(J)| = 2.

Plotting the graph:



It’s code:

```
function SingularityAnalysis()
%%
============================================================
%   SINGULARITY ANALYSIS - Delta Robot
%   - Find poses where robot is at singular configuration
%   - Singularity = det(J) ≈ 0
%%
============================================================

clear; clc; close all;
global r R l L

% Robot Parameters
R = 107.5;
r = 32.5;
L = 100;
l = 180;

fprintf('=== Singularity Analysis ===\n');
fprintf('Scanning for singular configurations...\n\n');

%% STEP 1: Scan joint space and check det(J)
th_range = -30:3:90;   % step = 3 deg

% Storage
pts_normal   = [];   % normal points
pts_singular = [];   % singular points
det_vals     = [];   % det values for all points

SINGULAR_THRESHOLD = .5;   % if |det(J)| < threshold => singular

count_n = 0;
count_s = 0;

for th1 = th_range
   for th2 = th_range
      for th3 = th_range

         [X, Y, Z, fl] = FKinem(th1, th2, th3);
```

```
        if fl == 0

            % Calculate Jacobian determinant
            detJ = calcDetJ(th1, th2, th3);

            if abs(detJ) < SINGULAR_THRESHOLD
                % SINGULAR point
                count_s = count_s + 1;
                pts_singular(count_s, :) = [X, Y, Z];

                fprintf('Singular Config #%d:\n', count_s);
                fprintf('  Angles: th1=%.1f, th2=%.1f, th3=%.1f deg\n',
th1,th2,th3);
                fprintf('  Position: (%.1f, %.1f, %.1f) mm\n', X, Y, Z);
                fprintf('  |det(J)| = %.4f\n\n', abs(detJ));

            else
                % Normal point
                count_n = count_n + 1;
                pts_normal(count_n, :) = [X, Y, Z];
                det_vals(count_n) = abs(detJ);
            end

        end
    end
  end
end

fprintf('Normal points   : %d\n', count_n);
fprintf('Singular points : %d\n\n', count_s);

%% STEP 2: Plot Results
figure('Name', 'Singularity Analysis', 'Position', [100 100 900 700]);

% Plot normal workspace (grey)
if ~isempty(pts_normal)
  scatter3(pts_normal(:,1), pts_normal(:,2), pts_normal(:,3), ...
        3, [0.7 0.7 0.7], 'filled');
  hold on;
end

% Plot singular points (red - big markers)
```

```
if ~isempty(pts_singular)
    scatter3(pts_singular(:,1), pts_singular(:,2), pts_singular(:,3), ...
            40, 'r', 'filled', 'MarkerEdgeColor', 'k');
end

xlabel('X (mm)'); ylabel('Y (mm)'); zlabel('Z (mm)');
title('Singularity Analysis - Red = Singular Configurations');
legend({'Normal Workspace', 'Singular Points'}, 'Location', 'best');
grid on; axis equal;
view(45, 25);
colormap(jet);

%% STEP 3: 2D Cross-section (top view XY)
figure('Name', 'Singularity - Top View (XY plane)', 'Position', [100 100 600
600]);

if ~isempty(pts_normal)
    scatter(pts_normal(:,1), pts_normal(:,2), ...
            3, [0.7 0.7 0.7], 'filled');
    hold on;
end

if ~isempty(pts_singular)
    scatter(pts_singular(:,1), pts_singular(:,2), ...
            40, 'r', 'filled', 'MarkerEdgeColor', 'k');
end

xlabel('X (mm)'); ylabel('Y (mm)');
title('Top View - Singular Regions (Red)');
legend({'Normal', 'Singular'}, 'Location', 'best');
grid on; axis equal;

%% STEP 4: Condition Number along a line
fprintf('=== Condition Number along Z-axis (X=0, Y=0) ===\n');

z_range = -150:-5:-350;
cond_vals = zeros(size(z_range));

figure('Name', 'Condition Number vs Z', 'Position', [100 100 700 400]);

for i = 1:length(z_range)
    Z_test = z_range(i);
```

```
    [th1,th2,th3,fl] = IKinem(0, 0, Z_test);

    if fl == 0
        detJ = calcDetJ(th1, th2, th3);
        cond_vals(i) = abs(detJ);
    else
        cond_vals(i) = nan;
    end
end

plot(z_range, cond_vals, 'b-', 'LineWidth', 2);
hold on;
yline(SINGULAR_THRESHOLD, 'r--', 'LineWidth', 2, 'Label', 'Singularity
Threshold');
xlabel('Z position (mm)');
ylabel('|det(J)|');
title('Manipulability along Z-axis (X=0, Y=0)');
grid on;

fprintf('\n=== DONE ===\n');

end
%%
% ============================================================
%   Helper: Calculate det(J) using numerical Jacobian
%%
% ============================================================
function detJ = calcDetJ(th1, th2, th3)

    delta = 0.01;

    [X0, Y0, Z0, fl0] = FKinem(th1, th2, th3);
    if fl0 ~= 0; detJ = 0; return; end

    [X1, Y1, Z1, fl1] = FKinem(th1+delta, th2, th3);
    [X2, Y2, Z2, fl2] = FKinem(th1, th2+delta, th3);
    [X3, Y3, Z3, fl3] = FKinem(th1, th2, th3+delta);

    if fl1~=0 || fl2~=0 || fl3~=0
        detJ = 0; return;
    end
```

```
    J = [ (X1-X0)/delta, (X2-X0)/delta, (X3-X0)/delta ;
          (Y1-Y0)/delta, (Y2-Y0)/delta, (Y3-Y0)/delta ;
          (Z1-Z0)/delta, (Z2-Z0)/delta, (Z3-Z0)/delta ];

    detJ = det(J);

end
```

Forward Kinematics singularity analysis equation:

$$t = R - r$$

$$y1 = -(t + L\cos\theta1), z1 = -L\sin\theta1, x1 = 0$$

$$y2 = (t + L\cos\theta2)\sin(\pi/6), \quad x2 = y2\tan(\pi/3), \quad z2 = -L\sin\theta2$$

$$y3 = (t + L\cos\theta3)\sin(\pi/6), \quad x3 = -y3\tan(\pi/3), \quad z3 = -L\sin\theta3$$

$$wi = xi2 + yi2 + zi2, \quad i = 1,2,3$$

$$dnm = (y2 - y1)x3 - (y3 - y1)x2$$

$$a1 = (z2 - z1)(y3 - y1) - (z3 - z1)(y2 - y1)$$

$$b1 = -((w2 - w1)(y3 - y1) - (w3 - w1)(y2 - y1))/2$$

$$a2 = -(z2 - z1)x3 + (z3 - z1)x2$$

$$b2 = \big((w2 - w1)x3 - (w3 - w1)x2\big)/2$$

$$aZ^2 + bZ + c = 0$$

$$a = a1^2 + a2^2 + dnm^2$$

$$b = 2(a1b1 + a2(b2 - y1 \cdot dnm) - z1 \cdot dnm^2)$$

$$c = (b2 - y1 \cdot dnm)^2 + b1^2 + dnm^2(z1^2 - l^2)$$

$$\boldsymbol{\Lambda} = b^2 - 4ac$$

$$Z = -\frac{b + \sqrt{\boldsymbol{\Lambda}}}{2a}, X = \frac{a1.Z + b1}{dnm}, Y = \frac{a2.Z + b2}{dnm}$$

Inverse kinematics:

$$\theta1 = f(X, Y, Z)$$

$$\theta2 = f(X\mathbf{cos}(2\pi/3)) + Y\mathbf{sin}(2\pi/3), Y\mathbf{cos}(2\pi/3) - X\mathbf{sin}(2\pi/3), Z)$$

$$y1^* = -R, \quad y0^* = y0 - r$$

$$a = \frac{\boldsymbol{x0^2} + y0^{*2} + z0^2 + L^2 - l^2 - y1^{*2}}{2.z0}$$

$$b = \frac{y1^* - y0^*}{z0}$$

$$D = -(a + b.y1*)^2 + L^2(b^2 + 1)$$

$$Yj = \frac{y1^* - ab - \sqrt{D}}{b^2 + 1}, Zj = a + b.Yj$$

$$\boldsymbol{\theta} = \mathbf{tan^{-1}}\left(\frac{-Zj}{y1^* - Yj}\right) + \begin{cases}\boldsymbol{\pi} \\ 0\end{cases} if Yj > y1*$$

$$D = -(a + b.y1^*)^2 + L^2(b^2 + 1) < 0$$